

CS571 Programming Languages

Spring 2026

8. Functional Programming I: Lambda Calculus

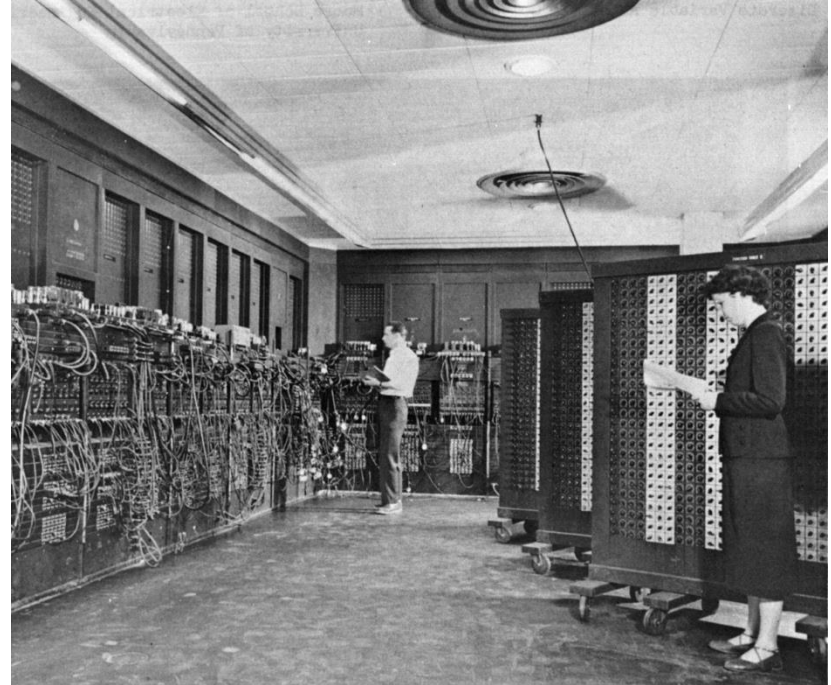
History

- What is computation?
 - Combinatory logic (1920's, [Moses Schönfinkel](#) and [Haskell Curry](#))
 - ***Lambda calculus*** (1930's, [Alonzo Church](#))
 - Turing machine (1930's, [Alan Turing](#))
 - ...

History

- First Turing complete, electronic general-purpose computer:
 - ENIAC (1945)
 - Designed by Mauchly and Eckert

Image from Wikipedia



Mathematical Function

- A function is a mapping from one set, called the **domain**, to another set, called the **range (codomain)**

Notations:

Domain

Range

$$+: (\mathbb{Z}, \mathbb{Z}) \mapsto \mathbb{Z}$$

$$\text{isNegative}: \mathbb{Z} \mapsto \{\text{True}, \text{False}\}$$

Infix form

Function application

$$+ \ 1 \ 2 = 3 \text{ (same as } 1+2=3\text{)}$$

$$\text{isNegative } 1 = \text{False}$$

The λ -Calculus



A pure λ -term is defined inductively as follows:

- Any variable $x \in \text{Var}$ is a λ -term
- If T is a λ -term, so is $\lambda x. T$ (**abstraction**)
- If T_1, T_2 are λ -terms, so is $T_1 T_2$ (**application**)

Analogy in C

Abstraction: `int f (int x) {return x+1;}`

Application: `f(2)`

λ -Term Examples

- Variables: $a, b, c, \dots, x, y, z \in \text{Var}$
- Function: $\lambda x. x$
 - Identity function $\lambda x. x$
 - A function that takes a parameter x and returns x
 - Constant function $\lambda x. y$
 - A function that takes a parameter x , ignores it, and returns y
- Application: $(\lambda x. x) y$
 - apply identity function to parameter y

How to Parse a λ -Term

- λ binding extends to the rightmost part
 - $\lambda x.x \ \lambda y.y \ z$ is parsed as $\lambda x.(x \ (\lambda y.(y \ z)))$
- Applications are left-associative
 - $T_1 \ T_2 \ T_3$ is parsed as $(T_1 \ T_2) \ T_3$

Number of Parameters

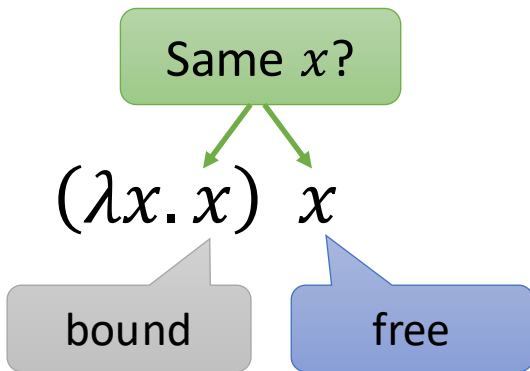
- In pure λ -calculus, λ only binds one parameter
- For convenience, we write $\lambda x y.T$ as a **shorthand (syntactic sugar)** for $\lambda x.(\lambda y.T)$
 - This process of removing parameters is called currying, which we will cover later in this course.

Syntax vs. Semantics

- Syntax: the structure/form of lambda terms
 - $T ::= V \mid T_1 T_2 \mid \lambda V. T$
 - $V ::= a \mid b \mid c \mid \dots \mid z$
 - () can be used for disambiguation
- Semantics: the meaning of lambda terms
 - Lambda calculus defines computation
 - What computation is defined by T ?

Capture-Avoiding Substitution

- In λ -calculus, computation is defined by **capture-avoiding substitution**
 - $T\{y/x\}$ means substitute all free x in T with y



Analogy in C

```
int x;  
int f(int x){return x;}
```

Bound vs. Free Variables

- In $(\lambda x. T)$, the variable x in T is bound to λx (c.f. local var)
- A variable is free if it is not bound to any λ (c.f. global var)
- A variable is bound to the closest λ
- Examples
 - $(\lambda x. x) x$
 - This applies the identity function to x (i.e., the x after dot is bound to λ)
 - $\lambda x. \lambda x. x$
 - This is a function that takes a parameter, and returns the identity function (i.e., the inner-most x is bound to the second λ)

Formal Definition

- Systematically, we define free variables as follows:
 - $FV(x) = \{x\}$
 - $FV(T_1 \ T_2) = FV(T_1) \cup FV(T_2)$
 - $FV(\lambda x. T) = FV(T) - \{x\}$
- Let $Var(T)$ be all variables used in T , the bound variables in T (written $BD(T)$) are
 - $BD(T) = Var(T) - FV(T)$

Examples

- $FV((\lambda x.x) x)$
- $FV(x) = \{x\}$
- $FV(T_1 T_2) = FV(T_1) \cup FV(T_2)$
- $FV(\lambda x.T) = FV(T) - \{x\}$

Examples

- $\text{FV}(\lambda y. (\lambda x. (x \ y)) \ x \)$
- $\text{FV}(x) = \{x\}$
- $\text{FV}(T_1 \ T_2) = \text{FV}(T_1) \cup \text{FV}(T_2)$
- $\text{FV}(\lambda x. T) = \text{FV}(T) - \{x\}$

Capture-Avoiding Substitution

- $T\{y/x\}$: substitute all free x in T with y when y is not captured in T (i.e., there is no λy in T)
- Example
 - $x\{y/x\} = ?$
 - $(x\ x)\{y/x\} = ?$
 - $(\lambda x. x)\{y/x\} = ?$
 - $(\lambda y. x)\{y/x\} = ?$

Capture-Avoiding Substitution

- $T\{y/x\}$: substitute all free x in T with y when y is not captured in T (i.e., there is no λy in T)
- Example
 - $x\{y/x\} = y$
 - $(x\ x)\{y/x\} = y\ y$
 - $(\lambda x. x)\{y/x\} = (\lambda x. x)$
 - $(\lambda y. x)\{y/x\} \neq (\lambda y. y)$

α -Reduction (Informal)

- $\lambda x. x$ defines the same function as $\lambda y. y$ and $\lambda z. z$ etc.
- Observation: the name of a parameter is **irrelevant**, hence can be changed
- Also called α -renaming

Analogy in C

The function	<code>int f (int x) { return x; }</code>
Is the same as	<code>int f (int y) { return y; }</code>

α -Reduction

- Are the following equivalent under α -renaming?

? $\lambda x. x x = \lambda y. y y$

? $\lambda x. x x = \lambda y. x y$

? $x = y$

? $\lambda x. \lambda x. x = \lambda y. \lambda x. y$

α -Reduction (formal)

- $\lambda x.T = \lambda y. T\{y/x\}$ when $y \notin FV(T)$ and y will not be captured/bound in T (there is no λy in T)

- Example

$$(\lambda x.x x) =_{\alpha}$$

α -Reduction (formal)

- $\lambda x.T = \lambda y.T\{y/x\}$ when $y \notin FV(T)$ and y will not be captured/bound in T (there is no λy in T)

- Example

$$(\lambda x. \lambda x. x) =_{\alpha}$$

α -Reduction (formal)

- $\lambda x.T = \lambda y.T\{y/x\}$ when $y \notin FV(T)$ and y will not be captured/bound in T (there is no λy in T)
 - Example
$$(\lambda x. \lambda y. x) =_{\alpha}$$
- Subtle point: what if y is captured in T ?
 - Use α -reduction to rename the captured y in T

β -Reduction (Informal)

- Identity function: $\lambda x. x$
- $(\lambda x. x) y = y$
- Observation: we can substitute the formal parameter with the true parameter

Analogy in C

Abstraction: `int f (int x) { return x; }`

Application: `f(y)` evaluates to `y`

β -Reduction

- The key reduction rule in λ calculus
- $(\lambda x. T_1) T_2 = T_1\{T_2/x\}$  Capture-avoiding substitution
- Substitution: replace all free x in T_1 with T_2
- Capture-avoiding:
 - Any free variables in $FV(T_2)$ will not be captured by any λ in T_1
 - If a capture would happen, use α -reduction to rename those λ in T_1

β -Reduction

Example 1: $(\lambda x. x x) y$

β -reduction

$$(\lambda x. T_1) T_2 = T_1\{T_2/x\}$$

β -Reduction

Example 2: $(\lambda x. \lambda y. x y) y$

β -reduction

$$(\lambda x. T_1) T_2 = T_1\{T_2/x\}$$

Useful Rule

- We say a term t is closed if it contains no free variable
 - Examples: $(\lambda x. x)$ $(\lambda x. \lambda y. x y)$
- Rule: for a capture avoiding substitution $T_1\{T_2/x\}$, the subtle captured-variable check is vacuously true when T_2 is closed

Another Useful Rule

- $(\lambda x_1 \dots x_n. T) T_1 \dots T_n = T\{T_1/x_1\} \dots \{T_n/x_n\}$
when $T_1 \dots T_n$ are all closed terms.

Evaluation

- An evaluation of a λ term is a sequence

$$T_1 = T_2 = T_3 = \dots$$

where each step is either an α -reduction or a β -reduction

Evaluation Order

- No reduction order is specified in classical λ -Calculus
- Some reduction sequences may never terminate
 - $(\lambda x. x x) (\lambda x. x x)$
 - This is called the Ω combinator, useful for modeling infinite loops and recursion!
 - For more detail, read [this article](#)

Evaluation Order

- **Church-Rosser Theorem**: if evaluation terminates, any order gives same result

$$\begin{array}{l} (\lambda x. (\lambda y. x) z) u \\ = \end{array}$$

$$\begin{array}{l} (\lambda x. (\lambda y. x) z) u \\ = \end{array}$$

Evaluation Order

- **Church-Rosser Theorem**: if evaluation terminates, any order gives same result

$$\begin{aligned} & (\lambda x. (\lambda y. x) z) u \\ &= (\lambda y. u) z \\ &= u \end{aligned}$$

$$\begin{aligned} & (\lambda x. (\lambda y. x) z) u \\ &= (\lambda x. x) u \\ &= u \end{aligned}$$

Examples

- $(\lambda x. x) x$

Examples

$$\begin{aligned} & \bullet (\lambda x \ y. x \ y) \ x \\ &= (\lambda x. (\lambda y. x \ y)) \ x \\ &= (\lambda y. x \ y) \end{aligned}$$

Examples

$$\begin{aligned} & \bullet (\lambda x \ y. x \ y) \ x \ y \\ &= (\lambda x. (\lambda y. x \ y)) \ x \ y \\ &= ((\lambda x. (\lambda y. x \ y)) \ x) \ y \\ &= (\lambda y. x \ y) \ y \\ &= x \ y \end{aligned}$$

Computational Power

Is the λ -Calculus Turing complete?

Yes!

How to encode things in λ -Calculus

- Booleans
 - **True, False**
 - Boolean operations: **AND, OR ...**
 - if expression: **IF** cond **THEN** expr **ELSE** expr
- Natural Numbers
 - Natural numbers 0, 1, 2, ...
 - Operations on natural numbers, +, -, *, /
- Structures
 - Pair, and left/right selection
 - List
- Recursive functions

Encoding: Boolean

- How to encode primitive values true/false?
 - Use functions
- What functions to choose?

$\text{TRUE} \triangleq \lambda x y. x$

$\text{FALSE} \triangleq \lambda x y. y$

Shorthand for $\lambda x. (\lambda y. x)$

Encoding: Boolean

$$\text{TRUE} \triangleq \lambda x y. x$$

$$\text{FALSE} \triangleq \lambda x y. y$$

Encoding of “IF”?

Hint: think of “IF” as a function

Goal:
$$\text{IF } b \ t \ f = \begin{cases} t & \text{when } b \text{ is TRUE} \\ f & \text{when } b \text{ is FALSE} \end{cases}$$

Encoding: Boolean

$$\begin{aligned}\text{TRUE} &\triangleq \lambda x y. x & \text{FALSE} &\triangleq \lambda x y. y \\ \text{IF} &\triangleq \lambda b t f. (b t f)\end{aligned}$$

Check: **IF TRUE** $t f =_{\beta} t$

Encoding: Boolean

$$\text{TRUE} \triangleq \lambda x y. x$$

$$\text{FALSE} \triangleq \lambda x y. y$$

Encoding of “AND”?

Goal: $\text{AND } b_1 b_2 = \begin{cases} \text{TRUE} & \text{when } b_1, b_2 \text{ are both TRUE} \\ \text{FALSE} & \text{otherwise} \end{cases}$

Encoding: Boolean

$\text{TRUE} \triangleq \lambda x y. x$ $\text{FALSE} \triangleq \lambda x y. y$
 $\text{AND} \triangleq \lambda b_1 b_2. (b_1 (b_2 \text{ TRUE } \text{FALSE}) \text{ FALSE})$

Check: $\text{AND } \text{TRUE } \text{FALSE} =_{\beta} \text{FALSE}$

Remark

Encoding is not unique

- Many encodings are equivalent
- $\text{AND} \triangleq \lambda b_1 b_2 . \text{IF } b_1 (\text{IF } b_2 \text{ TRUE FALSE}) \text{ FALSE}$
- $\text{NOT} \triangleq \lambda b . \text{IF } b \text{ FALSE TRUE}$

Encoding: Natural Numbers

Church numerals: $\underline{n} \triangleq \lambda f\ z. f^n\ z$

n -fold composition

Notation: $\underline{n}$ is the encoding of number n

- $\underline{0} \triangleq \lambda f\ z. f^0\ z = \lambda f\ z. z$
- $\underline{1} \triangleq \lambda f\ z. f^1\ z = \lambda f\ z. (f\ z)$
- $\underline{2} \triangleq \lambda f\ z. f^2\ z = \lambda f\ z. (f\ (f\ z))$

Encoding: Natural Numbers

$$\underline{n} \triangleq \lambda f \ z. f^n \ z$$

Encoding of “+ 1”?

Goal: $\text{SUCC } \underline{n} = \underline{n+1} = \lambda f \ z. f^{n+1} \ z$

Encoding: Natural Numbers

$$\begin{aligned}\underline{n} &\triangleq \lambda f z. f^n z \\ \text{SUCC} &\triangleq \lambda n f z. (f (n f z))\end{aligned}$$

Check: $\text{SUCC } \underline{1} = \underline{2}$

Encoding: Natural Numbers

$$\underline{n} \triangleq \lambda f z. f^n z$$
$$\text{SUCC} \triangleq \lambda n f z. (f (n f z))$$

Encoding of “+”?

Goal: PLUS $\underline{n} \underline{m} = \underline{n+m} = \lambda f z. f^{n+m} z$

Hint: $n + m$ is $m + 1 + 1 + \dots + 1$ (“+1” n times)

Encoding: Natural Numbers

$$\begin{aligned}\underline{n} &\triangleq \lambda f\ z. f^n\ z \\ \text{SUCC} &\triangleq \lambda n\ f\ z. (f\ (n\ f\ z)) \\ \text{PLUS} &\triangleq \lambda n\ m. (n\ \text{SUCC}\ m)\end{aligned}$$

Check: PLUS 1 2 = 3

Encoding: Pair and Selection

How to encode a data structure: PAIR?

- Hint: think of the functionality that PAIR supports
- Construct a PAIR object:
 - $\text{PAIR}(L, R) = p$
- Extract elements from a PAIR object:
 - $\text{LEFT } p = L$
 - $\text{RIGHT } p = R$

Encoding: Pair and Selection

$$\text{PAIR} \triangleq \lambda x y. \lambda f. f x y$$
$$\text{LEFT} \triangleq \lambda p. p \lambda x y. x$$
$$\text{RIGHT} \triangleq \lambda p. p \lambda x y. y$$

Check:

$p = \text{PAIR } L \ R = ?$

Encoding: Pair and Selection

$$\text{PAIR} \triangleq \lambda x y. \lambda f. f x y$$
$$\text{LEFT} \triangleq \lambda p. p \lambda x y. x$$
$$\text{RIGHT} \triangleq \lambda p. p \lambda x y. y$$

Check: $p = \text{PAIR } L R = \lambda f. f L R$

- $\text{LEFT } p =$

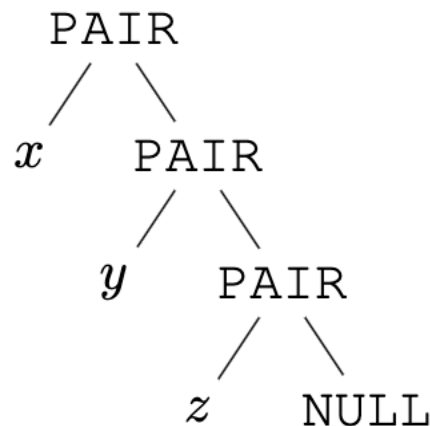
- $\text{RIGHT } p =$

Encoding: List

How to encode a list, say $[x, y, z]$?

Do this recursively

- Each non-empty list has a head and a tail
 - The head is an element
 - The tail is another list
- The empty list doesn't have head or tail



Encoding: List

- $\text{NULL} \triangleq \text{FALSE} \triangleq \lambda x y. y$
- $\text{CONS} \triangleq \text{PAIR} \triangleq \lambda x y. \lambda f. f x y$
- $\text{CAR} \triangleq \text{LEFT} \triangleq \lambda p. p \lambda x y. x$
- $\text{CDR} \triangleq \text{RIGHT} \triangleq \lambda p. p \lambda x y. y$



List Examples

- The list that has a single term `[TRUE]` ?
- `CONS TRUE NULL`
- `CAR (CONS TRUE NULL)`
- `CDR (CONS TRUE NULL)`

List Examples

- The list that has two terms `[FALSE, TRUE]` ?
- `CONS FALSE (CONS TRUE NULL)`
- `CAR (CONS FALSE (CONS TRUE NULL))`
- `CDR (CONS FALSE (CONS TRUE NULL))`

List Examples

- $\text{NULL} \triangleq \text{FALSE} \triangleq \lambda x y. y$
- $\text{CONS} \triangleq \text{PAIR} \triangleq \lambda x y. \lambda f. f x y$
- $\text{CAR} \triangleq \text{LEFT} \triangleq \lambda p. p \lambda x y. x$
- $\text{CDR} \triangleq \text{RIGHT} \triangleq \lambda p. p \lambda x y. y$

- How to check if a list is empty?
- $\text{ISNULL} \triangleq ?$

List Examples

- $\text{NULL} \triangleq \text{FALSE} \triangleq \lambda x y. y$
- $\text{CONS} \triangleq \text{PAIR} \triangleq \lambda x y. \lambda f. f x y$
- $\text{CAR} \triangleq \text{LEFT} \triangleq \lambda p. p \lambda x y. x$
- $\text{CDR} \triangleq \text{RIGHT} \triangleq \lambda p. p \lambda x y. y$

- How to check is a list is empty?
- $\text{ISNULL} \triangleq \lambda l. l (\lambda h t. \lambda d. \text{FALSE}) \text{TRUE}$

List Examples

- $\text{NULL} \triangleq \text{FALSE} \triangleq \lambda x y. y$
- $\text{CONS} \triangleq \text{PAIR} \triangleq \lambda x y. \lambda f. f x y$
- $\text{CAR} \triangleq \text{LEFT} \triangleq \lambda p. p \lambda x y. x$
- $\text{CDR} \triangleq \text{RIGHT} \triangleq \lambda p. p \lambda x y. y$
- $\text{ISNULL} \triangleq \lambda l. l (\lambda h t. \lambda d. \text{FALSE}) \text{TRUE}$

- ISNULL NULL

List Examples

- $\text{NULL} \triangleq \text{FALSE} \triangleq \lambda x y. y$
- $\text{CONS} \triangleq \text{PAIR} \triangleq \lambda x y. \lambda f. f x y$
- $\text{CAR} \triangleq \text{LEFT} \triangleq \lambda p. p \lambda x y. x$
- $\text{CDR} \triangleq \text{RIGHT} \triangleq \lambda p. p \lambda x y. y$
- $\text{ISNULL} \triangleq \lambda l. l (\lambda h t. \lambda d. \text{FALSE}) \text{TRUE}$

- $\text{ISNULL} (\text{CONS TRUE NULL})$

Pure vs. Applied λ -Calculus

- Pure λ -Calculus: the calculus discussed so far
- Applied λ -Calculus:
 - Built-in values and data structures (e.g., 1, 2, 3, true, false, (1 2 3))
 - Built-in functions (e.g., +, *, /, and, or)
 - Named functions
 - Recursion

All features can be encoded in
pure λ -calculus!